

Working with JUnit

Objectives

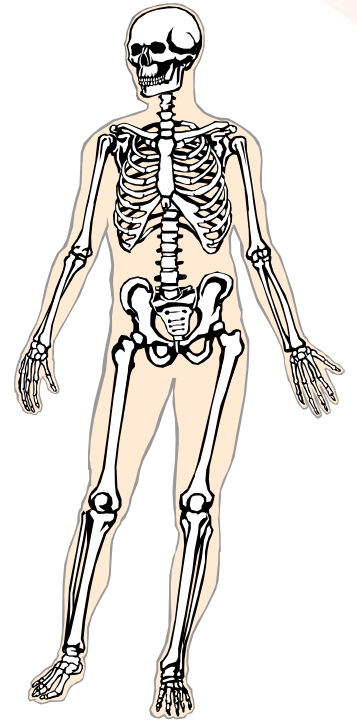
- Introduction to JUnit
- Building Test Cases
- Running Test Cases
- Maven Integration

Introduction

- JUnit testing is all about testing
 - Classes
 - Methods
- Test harnesses are created for this purpose
- JUnit is integrated into Eclipse, IntelliJ and Netbeans
- JUnit is also integrated with Maven and other integration and build tools

JUnit Test Anatomy

- A basic test class will
 - Define void methods annotated with **@Test** to contain the tests
 - To perform the check, each method will call **Assertions.assertXXX()** methods which will return true or false depending upon whether the test succeeds or fails



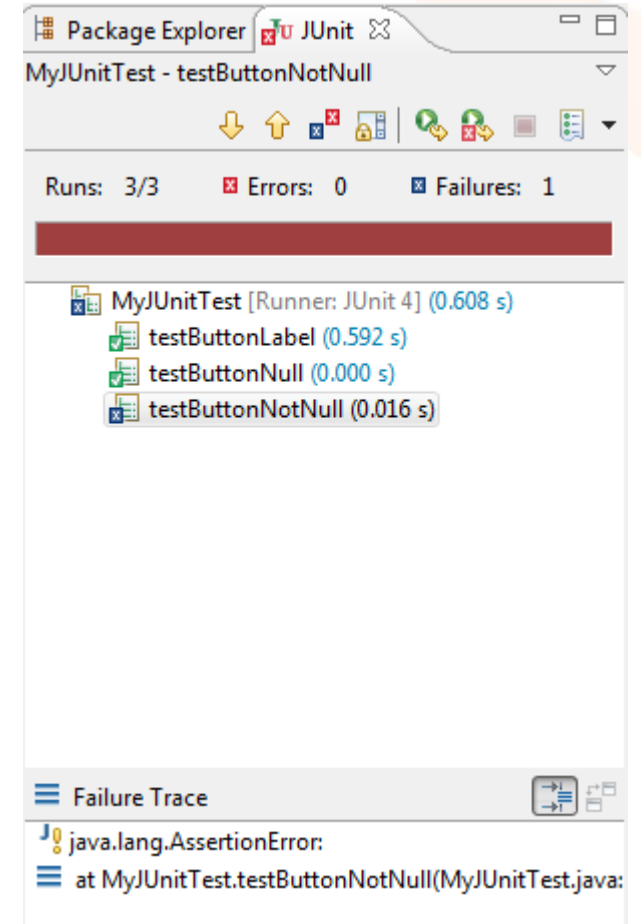
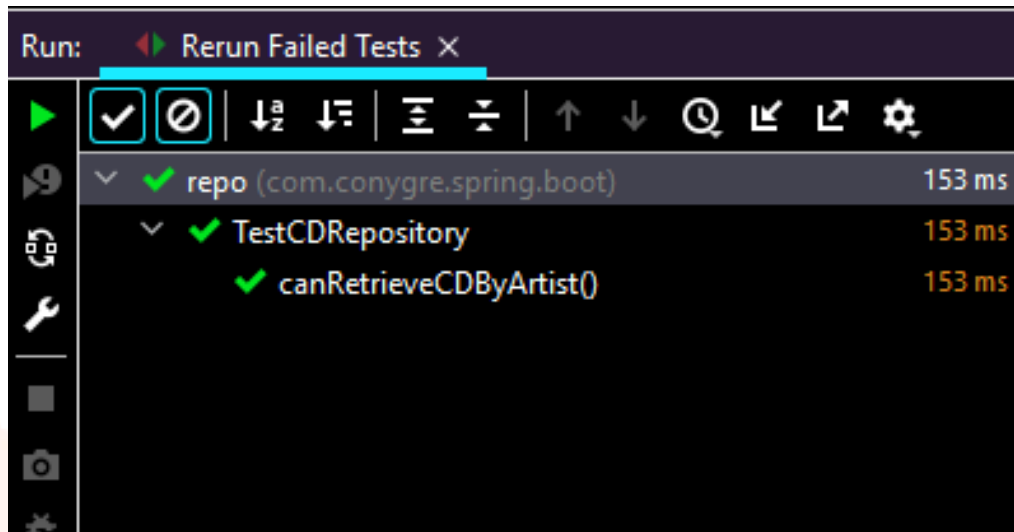
Basic JUnit Example

- Here is a basic JUnit Test Example
 - Note the three annotated **@Test** methods
 - Note the use of the **assertXXX()** methods

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import java.awt.Button;
public class MyJUnitTest {
    @Test // this test will pass
    public void testButtonLabel() {
        Button b1 = new Button("OK");
        Button b2 = new Button("OK");
        assertTrue(b1.getLabel().equals(b2.getLabel()));
    }
    @Test // this test will pass
    public void testButtonNull() {
        Button b1 = null;
        assertNull(b1);
    }
    @Test // this test will fail
    public void testButtonNotNull() {
        Button b1 = null;
        assertNotNull(b1);
    }
}
```

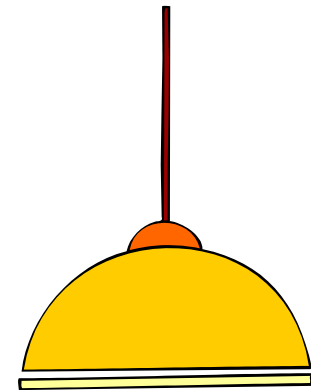
Running JUnit Tests with IDEs

- Developers can easily run tests using an IDE
 - In **Eclipse**, right click in the test code, click **Run As**, then click **JUnit Test**
 - Using IntelliJ you can use the Test console or simply click on the Play button next to your tests



Working with Fixtures

- In the previous example, the objects required by the tests were created for each test
 - These objects can be created to be shared by the tests, and are referred to as **fixtures**
- Fixtures are
 - Declared as instance variables
 - Initialised in a method annotated with **@BeforeEach**
 - Any cleanup is done in a method annotated with **@AfterEach**



Fixture Example

- Here is a Fixture JUnit Test Example
 - Note the instance variables or fixtures
 - Note the method annotated with **@BeforeEach**

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.BeforeEach;
import java.awt.Button;

public class MyJUnitTestWithFixtures {

    // the fixtures as instance variables
    private Button b1, b2, b3;

    @BeforeEach // set up the fixtures in this method
    public void setUp() {
        b1 = new Button("OK");
        b2 = new Button("OK");
    }

    @Test // this test should pass
    public void testButtonLabel() {
        assertTrue(b1.getLabel().equals(b2.getLabel()));
    }

    @Test // this test will fail
    public void testButtonNotNull() {
        assertNotNull(b3);
    }
}
```

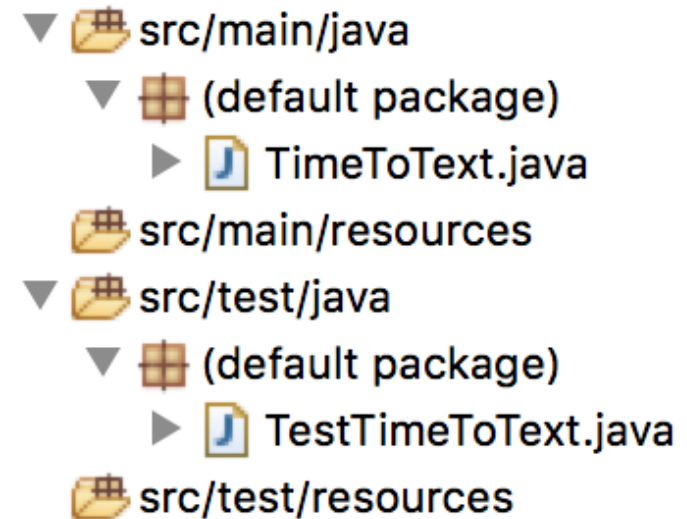

@BeforeAll and @AfterAll

- It is also possible to have methods that run once before all tests in a class or after all tests in a class
- The methods must be static and have the following annotations
 - @BeforeAll
 - @AfterAll

```
public class MyJUnitTestWithFixtures {  
    @BeforeAll  
    public static void doSomeOneOffSetup() {  
        // read a config file or something slimilar  
    }  
}
```

Maven / Gradle Integration

- Any JUnit tests in the src/test/java folder will be run as part of a Maven or Gradle build



Summary

- Introduction to JUnit
- Building Test Cases
- Running Test Cases
- Maven Integration